LLMs are Stateless

```python
"""
Stateless LLM demo

Shows that a raw LLM call has NO memory between invocations.
Each call is a fresh context — previous messages are discarded.
"""

from dotenv import load_dotenv

load_dotenv()

from langchain_openai import ChatOpenAI

MODEL = "gpt-4o"
llm = ChatOpenAI(model=MODEL)


def ask(messages: list[dict]) -> str:
    response = llm.invoke(messages)
    return response.content

def main() -> None:
    print("Type 'exit' to quit.\n")

    while True:
        user_input = input("You: ").strip()
        if user_input.lower() == "exit":
            break
        if not user_input:
            continue

        # Every call gets only the current message — no conversation history
        messages = [
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": user_input},
        ]
        reply = ask(messages)
        print(f"LLM: {reply}\n")

if __name__ == "__main__":
    main()
```

Short-Term Memory

https://docs.langchain.com/oss/python/langchain/short-term-memory

In RAM

```python
"""
Short-term memory demo
Conversation history is appended and sent every call
The history lives in-process (a plain list). It resets on restart.
"""

from dotenv import load_dotenv
load_dotenv()

from langchain_openai import ChatOpenAI
MODEL = "gpt-4o"
llm = ChatOpenAI(model=MODEL)

# In-memory history — grows with every turn
history: list[dict] = [
    {"role": "system", "content": "You are a helpful assistant."},
]

def ask(user_input: str) -> str:
    history.append({"role": "user", "content": user_input})
    response = llm.invoke(history)
    reply = response.content
    history.append({"role": "assistant", "content": reply})
    return reply

def main() -> None:
    print("Short-term memory demo — full history is sent on every call.")
    print("Type 'exit' to quit, '/history' to see stored messages.\n")

    while True:
        user_input = input("You: ").strip()
        if user_input.lower() == "exit":
            break
        if not user_input:
            continue
        if user_input.lower() == "/history":
            for msg in history:
                print(f"  [{msg['role']}] {msg['content'][:120]}")
```

```
            print()
            continue
        reply = ask(user_input)
        print(f"LLM: {reply}\n")

if __name__ == "__main__":
    main()
```

Persist in SQLite with Threads

```
"""
Short-term memory demo — SQLite persistence + threads

- history now survives restarts (stored in SQLite)
- multiple threads: each thread_id is an independent conversation

Commands:
 /history              show messages in current thread
 /threads              list all saved thread IDs
 /switch <thread_id>   switch to a different thread
 exit                  quit
"""

import sqlite3
from pathlib import Path

from dotenv import load_dotenv

load_dotenv()

from langchain_openai import ChatOpenAI

MODEL = "gpt-4o"
DB_PATH = Path("data/memory/shortterm_memory_demo.db")
llm = ChatOpenAI(model=MODEL)


# --- persistence helpers ---

def init_db(conn: sqlite3.Connection) -> None:
    conn.execute("""
        CREATE TABLE IF NOT EXISTS threads (
            thread_id TEXT NOT NULL,
            role      TEXT NOT NULL,
```

```
            content   TEXT NOT NULL
        )
    """)
    conn.commit()


def load_history(conn: sqlite3.Connection, thread_id: str) -> list[dict]:
    rows = conn.execute(
        "SELECT role, content FROM threads WHERE thread_id = ? ORDER BY rowid",
        (thread_id,),
    ).fetchall()
    history = [{"role": "system", "content": "You are a helpful assistant."}]
    history += [{"role": r, "content": c} for r, c in rows]
    return history


def save_message(conn: sqlite3.Connection, thread_id: str, role: str, content: str)
-> None:
    conn.execute(
        "INSERT INTO threads (thread_id, role, content) VALUES (?, ?, ?)",
        (thread_id, role, content),
    )
    conn.commit()


def list_threads(conn: sqlite3.Connection) -> list[str]:
    rows = conn.execute(
        "SELECT DISTINCT thread_id FROM threads ORDER BY thread_id"
    ).fetchall()
    return [r[0] for r in rows]


# --- agent ---
def ask(conn: sqlite3.Connection, thread_id: str, user_input: str) -> str:
    save_message(conn, thread_id, "user", user_input)
    history = load_history(conn, thread_id)
    reply = llm.invoke(history).content
    save_message(conn, thread_id, "assistant", reply)
    return reply

def main() -> None:
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(str(DB_PATH))
    init_db(conn)
```

```
    thread_id = "default"    print(f"Thread: {thread_id}  (use /switch <id> to
change)")
    print("Commands: /history, /threads, /switch <id>, exit\n")


    while True:
        user_input = input("You: ").strip()
        if not user_input:
            continue

        if user_input.lower() == "exit":
            break

        if user_input.lower() == "/history":
            for msg in load_history(conn, thread_id):
                print(f"  [{msg['role']}] {msg['content'][:120]}")
            print()
            continue

        if user_input.lower() == "/threads":
            threads = list_threads(conn)
            print("  Saved threads:", threads or "(none yet)")
            print(f"  Active: {thread_id}\n")
            continue

        if user_input.lower().startswith("/switch "):
            new_id = user_input.split(maxsplit=1)[1].strip()
            if not new_id:
                print("Usage: /switch <thread_id>\n")
                continue
            thread_id = new_id
            print(f"Switched to thread: {thread_id}\n")
            continue

        reply = ask(conn, thread_id, user_input)
        print(f"LLM: {reply}\n")

    conn.close()


if __name__ == "__main__":
    main()
```

Context Window Strategies

Sliding Window

https://docs.langchain.com/oss/python/langchain/short-term-memory#before-model
https://docs.langchain.com/oss/python/langchain/middleware/overview

```
"""
Short-term memory demo — Sliding Window

Uses @before_model middleware to trim history before every model call.
Keeps system message + last N messages. Older messages are dropped.

Tradeoff: simple and cheap, but early context is lost once the window fills.
"""

from pathlib import Path
from typing import Any

from dotenv import load_dotenv

load_dotenv()

from langchain.agents import AgentState, create_agent
from langchain.agents.middleware import before_model
from langchain.messages import RemoveMessage
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph.message import REMOVE_ALL_MESSAGES
from langgraph.runtime import Runtime

MODEL = "gpt-4o"
WINDOW = 4  # number of recent messages to keep
DB_PATH = Path("data/memory/sliding_window.db")

@before_model
def sliding_window(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    messages = state["messages"]
    if len(messages) <= WINDOW:
        return None

    recent = messages[-WINDOW:]
    return {
        "messages": [
            RemoveMessage(id=REMOVE_ALL_MESSAGES),
            *recent,
        ]
```

```
    }


def print_thread_state(agent, thread_id: str) -> None:
    snapshot = agent.get_state({"configurable": {"thread_id": thread_id}})
    messages = (snapshot.values or {}).get("messages", [])
    print(f"\n  thread: {thread_id} | stored messages: {len(messages)}")
    for msg in messages:
        role = getattr(msg, "type", "unknown")
        content = str(getattr(msg, "content", ""))[:120]
        print(f"  [{role}] {content}")
    print()


def list_threads(checkpointer: SqliteSaver) -> list[str]:
    checkpointer.setup()
    rows = checkpointer.conn.execute(
        "SELECT DISTINCT thread_id FROM checkpoints ORDER BY thread_id"
    ).fetchall()
    return [r[0] for r in rows]


def main() -> None:
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)

    with SqliteSaver.from_conn_string(str(DB_PATH)) as checkpointer:
        agent = create_agent(
            model=f"openai:{MODEL}",
            tools=[],
            system_prompt="You are a helpful assistant.",
            middleware=[sliding_window],
            checkpointer=checkpointer,
        )

        thread_id = "default"
        config = {"configurable": {"thread_id": thread_id}}

        print(f"Sliding window demo — keeping last {WINDOW} messages.")
        print("Try recalling something said early on after several turns.")
        print("Commands: /history, /threads, /switch <id>, exit\n")

        while True:
            user_input = input("You: ").strip()
            if not user_input:
                continue
```

```
            if user_input.lower() == "exit":
                break
            if user_input.lower() == "/history":
                print_thread_state(agent, thread_id)
                continue
            if user_input.lower() == "/threads":
                threads = list_threads(checkpointer)
                print(f"  Saved threads: {threads or '(none yet)'}")
                print(f"  Active: {thread_id}\n")
                continue
            if user_input.lower().startswith("/switch "):
                new_id = user_input.split(maxsplit=1)[1].strip()
                if not new_id:
                    print("Usage: /switch <thread_id>\n")
                    continue
                thread_id = new_id
                config = {"configurable": {"thread_id": thread_id}}
                print(f"Switched to thread: {thread_id}\n")
                continue

            result = agent.invoke(
                {"messages": [{"role": "user", "content": user_input}]},
                config,
            )
            print(f"LLM: {result['messages'][-1].content}\n")


if __name__ == "__main__":
    main()
```

Summarization

https://docs.langchain.com/oss/python/langchain/short-term-memory#summarize-messages

```
"""
Short-term memory demo — Summarization

Custom @before_model middleware that compresses older messages once the
message count threshold is reached. Recent messages are kept raw alongside
the summary. Prints when summarization is triggered.

Tradeoff: retains gist of early context, but fine details may be lost.

Commands:
```

```
 /history              show stored thread state
 /threads              list all saved thread IDs
 /switch <thread_id>   switch to a different thread
 exit                  quit
"""

from pathlib import Path
from typing import Any

from dotenv import load_dotenv

load_dotenv()

from langchain.agents import AgentState, create_agent
from langchain.agents.middleware import before_model
from langchain.messages import RemoveMessage
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph.message import REMOVE_ALL_MESSAGES
from langgraph.runtime import Runtime

MODEL = "gpt-4o"
SUMMARIZE_AFTER = 6  # summarize when stored messages exceed this count
KEEP_RECENT = 4      # keep last N messages raw after summarization
DB_PATH = Path("data/memory/summarization.db")

summarizer = ChatOpenAI(model=MODEL)


@before_model
def summarize_middleware(state: AgentState, runtime: Runtime) -> dict[str, Any] |
None:
   messages = state["messages"]
   if len(messages) <= SUMMARIZE_AFTER:
       return None

   older = messages[:-KEEP_RECENT]
   recent = messages[-KEEP_RECENT:]

   conversation_text = "\n".join(
       f"{getattr(m, 'type', 'unknown')}: {getattr(m, 'content', '')}"
       for m in older
   )
   summary = summarizer.invoke(
```

```
        f"Summarize this conversation in 2-3 sentences, preserving key
facts:\n\n{conversation_text}"
    ).content

    print(f"\n  [summarization triggered — compressed {len(older)} messages into
summary]")
    print(f"  [summary: {summary[:120]}...]\n")

    return {
        "messages": [
            RemoveMessage(id=REMOVE_ALL_MESSAGES),
            {"role": "system", "content": f"Conversation summary so
far:\n{summary}"},
            *recent,
        ]
    }

def print_thread_state(agent, thread_id: str) -> None:
    snapshot = agent.get_state({"configurable": {"thread_id": thread_id}})
    messages = (snapshot.values or {}).get("messages", [])
    print(f"\n  thread: {thread_id} | stored messages: {len(messages)}")
    for msg in messages:
        role = getattr(msg, "type", "unknown")
        content = str(getattr(msg, "content", ""))[:120]
        print(f"  [{role}] {content}")
    print()

def list_threads(checkpointer: SqliteSaver) -> list[str]:
    checkpointer.setup()
    rows = checkpointer.conn.execute(
        "SELECT DISTINCT thread_id FROM checkpoints ORDER BY thread_id"
    ).fetchall()
    return [r[0] for r in rows]


def main() -> None:
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)

    with SqliteSaver.from_conn_string(str(DB_PATH)) as checkpointer:
        agent = create_agent(
            model=f"openai:{MODEL}",
            tools=[],
            system_prompt="You are a helpful assistant.",
            middleware=[summarize_middleware],
            checkpointer=checkpointer,
```

```
        )

        thread_id = "default"
        config = {"configurable": {"thread_id": thread_id}}

        print(f"Summarization demo — summarizes after {SUMMARIZE_AFTER} messages,
keeps last {KEEP_RECENT} raw.")
        print("Commands: /history, /threads, /switch <id>, exit\n")

        while True:
            user_input = input("You: ").strip()
            if not user_input:
                continue
            if user_input.lower() == "exit":
                break
            if user_input.lower() == "/history":
                print_thread_state(agent, thread_id)
                continue
            if user_input.lower() == "/threads":
                threads = list_threads(checkpointer)
                print(f"  Saved threads: {threads or '(none yet)'}")
                print(f"  Active: {thread_id}\n")
                continue
            if user_input.lower().startswith("/switch "):
                new_id = user_input.split(maxsplit=1)[1].strip()
                if not new_id:
                    print("Usage: /switch <thread_id>\n")
                    continue
                thread_id = new_id
                config = {"configurable": {"thread_id": thread_id}}
                print(f"Switched to thread: {thread_id}\n")
                continue

            result = agent.invoke(
                {"messages": [{"role": "user", "content": user_input}]},
                config,
            )
            print(f"LLM: {result['messages'][-1].content}\n")


if __name__ == "__main__":
    main()
```

Long-Term Memory

https://docs.langchain.com/oss/python/langchain/long-term-memory
https://docs.langchain.com/oss/python/langgraph/stores

Extract and Store User Profile

```
"""
Long-term memory
Introduces SqliteStore, LangGraph's durable key-value store for LTM.
No tools, no agent — just raw store.put / store.get / store.search.

Two primitives to learn:
 namespace  = tuple path (like a folder)  e.g. ("users",)
 key        = string within the namespace  e.g. "alice"
 value      = any JSON-serialisable dict

Architecture this file teaches:
   user input
       └─► LLM answers
       └─► second LLM call extracts facts
       └─► store.put(namespace, key, facts)   # persisted to SQLite

On restart:
   store.get(namespace, key)                  # load facts back
       └─► inject into system prompt
       └─► LLM answers with context
Demo:
 1. Tell the agent your name, job, and a preference.
 2. Type 'exit'.
 3. Rerun — the agent still knows you.
 4. Use '/facts' to inspect what is stored.
"""

import json
from pathlib import Path

from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langgraph.store.sqlite import SqliteStore

load_dotenv()
MODEL = "gpt-4o"
llm = ChatOpenAI(model=MODEL)
```

```python
# ---------------------------------------------------------------------------
# Storage layout
#   namespace ("users",)  key <user_id>  →  profile dict
#   namespace (<user_id>, "facts")  key <slug>  →  individual fact
# ---------------------------------------------------------------------------
USERS_NS = ("users",)
DB_PATH = Path(__file__).parent / "data" / "memory" / "ltm_store.db"
DB_PATH.parent.mkdir(parents=True, exist_ok=True)

USER_ID = "demo-user"

def facts_namespace(user_id: str) -> tuple:
    return (user_id, "facts")

# ---------------------------------------------------------------------------
# Fact extraction — a plain second LLM call, not a tool
# ---------------------------------------------------------------------------
def extract_facts(user_input: str, reply: str, existing: dict) -> dict:
    prompt = f"""You are a fact extractor.
Extract any durable facts about the user from this conversation turn
(name, job, preferences, location, constraints, goals — anything worth
remembering long-term).

Existing facts already stored:
{json.dumps(existing, indent=2)}

User said : {user_input}
Assistant : {reply}

Return a single JSON object that MERGES existing facts with any NEW facts
you found. If there is nothing new, return existing facts unchanged.
Return ONLY valid JSON — no markdown, no explanation."""

    result = llm.invoke([{"role": "user", "content": prompt}])
    try:
        return json.loads(result.content)
    except json.JSONDecodeError:
        return existing


def build_system_prompt(facts: dict) -> str:
    base = "You are a helpful assistant with long-term memory."
    if not facts:
        return base
```

```
    lines = "\n".join(f"  - {k}: {v}" for k, v in facts.items())
    return f"{base}\n\nWhat you know about the user:\n{lines}"


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main() -> None:
    # SqliteStore is a context manager — keeps the DB connection alive.
    with SqliteStore.from_conn_string(str(DB_PATH)) as store:
        store.setup()

        # Load existing facts for this user from LTM
        item = store.get(USERS_NS, USER_ID)
        facts: dict = dict(item.value) if item else {}

        history = [{"role": "system", "content": build_system_prompt(facts)}]

        print("Step 1 — raw SqliteStore (no tools, no agent).")
        print(f"DB: {DB_PATH}")
        print(f"user_id: {USER_ID}")
        if facts:
            print(f"Loaded {len(facts)} fact(s) from previous session.")
        print("Commands: '/facts'  'exit'\n")

        while True:
            user_input = input("You: ").strip()
            if user_input.lower() == "exit":
                break
            if not user_input:
                continue
            if user_input.lower() == "/facts":
                # store.get returns an Item; .value is the stored dict
                item = store.get(USERS_NS, USER_ID)
                print(json.dumps(dict(item.value) if item else {}, indent=2))
                print()
                continue

            history.append({"role": "user", "content": user_input})
            response = llm.invoke(history)
            reply = response.content
            history.append({"role": "assistant", "content": reply})
            print(f"LLM: {reply}\n")

            # Extract new facts and persist — this is what a tool will do in step 2
```

```
            updated = extract_facts(user_input, reply, facts)
            if updated != facts:
                facts = updated
                # store.put(namespace, key, value)
                # Calling put again on same key is an upsert — it overwrites.
                store.put(USERS_NS, USER_ID, facts)
                print(f"  [stored {len(facts)} fact(s) to SqliteStore]\n")


if __name__ == "__main__":
    main()
```

STM and LTM

```
"""
Two memory layers, two databases:
 thread_id  →  SqliteSaver  →  all messages in this chat thread (STM)
 user_id    →  SqliteStore  →  behavioral rules shared across threads (LTM)

Procedural memory = "how to behave" rules.
When the user says "always respond in bullet points" or "be very concise",
the agent detects this as a procedural rule, saves it to the store, and
applies it in every future session — including after a full restart.

Demo:
 1. Say "always respond in bullet points".
 2. Observe the agent adopts the behavior immediately.
 3. Type 'exit', rerun — the rule is still active.
 4. Use '/show-ltm' and '/show-stm' to inspect both memory layers.
 5. Run with --thread-id chat2 to start a new thread (STM resets, LTM carries
over).
"""

import argparse
import re
from dataclasses import dataclass
from pathlib import Path

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.tools import ToolRuntime, tool
```

```python
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.store.sqlite import SqliteStore

load_dotenv()

MODEL = "gpt-4o"

DB_DIR = Path(__file__).parent / "data" / "memory"
DB_DIR.mkdir(parents=True, exist_ok=True)

LTM_DB = str(DB_DIR / "ltm.db")
STM_DB = str(DB_DIR / "stm.db")



# ---------------------------------------------------------------------------
# Context — passed on every agent.invoke(..., context=Context(...))
# create_agent injects this into ToolRuntime so tools know which user to use.
# ---------------------------------------------------------------------------
@dataclass
class Context:
    user_id: str


def rules_ns(user_id: str) -> tuple:
    return (user_id, "rules")


# ---------------------------------------------------------------------------
# Tools — ToolRuntime[Context] gives each tool access to:
#   runtime.store   → the SqliteStore (LTM)
#   runtime.context → the Context dataclass (user_id)
# ---------------------------------------------------------------------------
@tool
def save_rule(rule: str, runtime: ToolRuntime[Context]) -> str:
    """Save a behavioral rule learned from the user.

    Call this whenever the user instructs you HOW to behave —
    tone, format, language, verbosity, etc.

    Args:
        rule: the behavioral instruction as a clear, imperative sentence
              e.g. 'Always respond in bullet points.'
    """
    user_id = runtime.context.user_id
    key = re.sub(r"[^a-z0-9]+", "-", rule.lower())[:48].strip("-") or "rule"
```

```
    runtime.store.put(rules_ns(user_id), key, {"rule": rule})
    return f"Saved procedural rule: {rule}"


@tool
def list_rules(runtime: ToolRuntime[Context]) -> str:
    """List all behavioral rules learned from the user."""
    user_id = runtime.context.user_id
    items = runtime.store.search(rules_ns(user_id), limit=50)
    if not items:
        return "No behavioral rules stored yet."
    return "\n".join(f"- {i.value['rule']}" for i in items)


TOOLS = [save_rule, list_rules]


# ---------------------------------------------------------------------------
# Inspect helpers
# ---------------------------------------------------------------------------
def list_threads(checkpointer: SqliteSaver) -> list[str]:
    checkpointer.setup()
    rows = checkpointer.conn.execute(
        "SELECT DISTINCT thread_id FROM checkpoints ORDER BY thread_id"
    ).fetchall()
    return [r[0] for r in rows]



def print_ltm(store: SqliteStore, user_id: str) -> None:
    store.setup()
    items = store.search(rules_ns(user_id), limit=50)
    print(f"\n=== LONG-TERM MEMORY (user_id={user_id}) ===")
    if not items:
        print("  (no rules stored yet)")
    for i in items:
        print(f"  • {i.value['rule']}")
    print()


def print_stm(agent, thread_id: str) -> None:
    snapshot = agent.get_state({"configurable": {"thread_id": thread_id}})
    messages = (snapshot.values or {}).get("messages", [])
    print(f"\n=== SHORT-TERM MEMORY (thread_id={thread_id}) ===")
    print(f"  {len(messages)} message(s) stored")
```

```
    for i, m in enumerate(messages, 1):
        role = getattr(m, "type", getattr(m, "role", "?"))
        text = str(getattr(m, "content", ""))[:120]
        print(f"  {i:02d}. [{role}] {text}")
    print()


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main() -> None:
    parser = argparse.ArgumentParser(description="create_agent + STM + LTM")
    parser.add_argument("--user-id", default="demo-user")
    parser.add_argument("--thread-id", default="thread-1")
    args = parser.parse_args()

    user_id = args.user_id
    thread_id = args.thread_id

    # thread_id routes STM (which conversation thread to restore)
    config = {"configurable": {"thread_id": thread_id}}
    # user_id routes LTM (which user's rules to load/save)
    context = Context(user_id=user_id)

    with SqliteStore.from_conn_string(LTM_DB) as store:
        store.setup()

        with SqliteSaver.from_conn_string(STM_DB) as checkpointer:

            # Load existing rules so the system prompt reflects them on startup
            existing_rules = store.search(rules_ns(user_id), limit=50)
            rules_text = (
                "\n".join(f"- {i.value['rule']}" for i in existing_rules)
                if existing_rules
                else "None yet."
            )

            system_prompt = f"""You are a helpful assistant that learns behavioral
preferences.

When the user tells you HOW to behave (format, tone, language, verbosity, style),
call save_rule to persist it as a long-term procedural memory.
At the start of each response, silently apply all active rules.
If asked what rules you follow, call list_rules.
```

```
Active rules from long-term memory:
{rules_text}"""

            # create_agent handles the tool loop automatically.
            # store=      wires LTM into tools via ToolRuntime[Context].
            # checkpointer= wires STM (full message history per thread_id).
            # context_schema= tells create_agent the shape of our Context dataclass.
            agent = create_agent(
                model=f"openai:{MODEL}",
                tools=TOOLS,
                system_prompt=system_prompt,
                store=store,
                checkpointer=checkpointer,
                context_schema=Context,
            )

            print("create_agent + SqliteSaver (STM) + SqliteStore (LTM).")
            print(f"LTM DB : {LTM_DB}")
            print(f"STM DB : {STM_DB}")
            print(f"user_id={user_id}  |  thread_id={thread_id}")
            if existing_rules:
                print(f"Loaded {len(existing_rules)} rule(s) from previous
sessions.")
            print("Commands: '/show-ltm'  '/show-stm'  '/threads'  '/switch
<thread-id>'  'exit'\n")

            while True:
                user_input = input("You: ").strip()
                if user_input.lower() == "exit":
                    break
                if not user_input:
                    continue
                if user_input.lower() == "/show-ltm":
                    print_ltm(store, user_id)
                    continue
                if user_input.lower() == "/show-stm":
                    print_stm(agent, thread_id)
                    continue
                if user_input.lower() == "/threads":
                    threads = list_threads(checkpointer)
                    print(f"  Saved threads: {threads or '(none yet)'}")
                    print(f"  Active: {thread_id}\n")
                    continue
                if user_input.lower().startswith("/switch "):
                    new_thread = user_input.split(maxsplit=1)[1].strip()
```

```
                    if not new_thread:
                        print("Usage: /switch <thread-id>\n")
                        continue
                    thread_id = new_thread
                    config = {"configurable": {"thread_id": thread_id}}
                    print(f"Switched to thread: {thread_id}  (LTM rules
unchanged)\n")
                    continue

                result = agent.invoke(
                    {"messages": [{"role": "user", "content": user_input}]},
                    config=config,
                    context=context,
                )
                reply = result["messages"][-1].content
                print(f"Agent: {reply}\n")


if __name__ == "__main__":
    main()
```

Caching

Prompt Caching

https://developers.openai.com/api/docs/guides/prompt-caching

https://platform.claude.com/docs/en/build-with-claude/prompt-caching

https://docs.langchain.com/oss/javascript/integrations/middleware/anthropic#full-example